



Fig. 4: Degree of power consumption for each example generic mode. Actual mode name and current consumption value depend on the clock configuration in each mode (activation of internal oscillators and PLL, selected frequencies, etc) based on different MCU vendors

battery life span.

Low-power mode types. The modes vary in degree to which the CPU is aware of its surroundings and the different clocks that the processor must keep running. Based on the clock supply by either PLL, main clock, on-chip RC oscillator or sub-clock, the degree of power consumption for each type of sleep mode varies.

The most common LPMs are idle mode, power-save (PS)/timer mode and power-down (PWD)/stop mode. Fig. 4 shows the active oscillator type and PLL clock source configuration in each mode.

Idle mode is a shallow sleep mode where only parts of the device are shut down but the main parts of the MCU are running.

In PS mode, everything is turned off, except a 32kHz clock running from a crystal to keep track of the time. It allows only the main clock, sub-clock oscillations, time-base timer or watch prescaler to work. The operating clock for the CPU and peripheral resources is stopped in this mode to reduce power consumption.

In PWD mode, everything, including the clock source, is shut down. All clocks are halted, but MCU status, RAM and register contents are preserved. This mode consumes the least

current and only external interrupts can wake up the MCU from PWD mode.

The advantage of having multiple sleep modes is the flexibility that it provides to shut down any part of the MCU that is not absolutely necessary to the function at hand. The amount of power that can be saved depends on the mode being used.

Clocking systems. In many applications, the time spent in active mode is insignificant compared to the time spent in PS mode with everything, except an RTC and the BOD, turned off. In low-power applications, the power consumed in PS mode (sum of current consumption of PWD, BOD and 32kHz oscillator) is the most significant contributor to overall power consumption. Therefore lowering the current consumption of the 32kHz oscillator can significantly reduce power consumption of the overall system. Designers should thoroughly evaluate the current consumed with the crystal oscillator running.

Applications may enter and exit various LPMs several hundred times a second. The ability to get into and out of the LPMs, and process data quickly, is crucial because the CPU waiting for the clock to become stable wastes current.

Most low-power MCUs have 'instant-on' clocks that are ready for the CPU in less than 10-20 μ s. But it is important to understand which clocks are instantly 'on' and which are not.

Some MCUs have a two-stage clock wake-up providing a low-frequency (usually 32,768Hz) clock to the CPU while a high-frequency clock is being stabilised, which can take up to a millisecond or longer. On these devices, the CPU may be operational within 10-15 μ s, but run at a low, inefficient frequency or an incorrect high-speed frequency. If the CPU needs to execute only a few instructions, say 25, it will take about 763 μ s at 32,768 Hz. The CPU consumes less current at a low frequency than it would at a high frequency, but not enough to make up for the difference in processing time with the low frequency.

Alternatively, the CPU may be running at a high but incorrect frequency. In this case, the processing time is low but the application cannot rely on the clock source for accurate timing. If accurate timing is required, the CPU must wait for it to become stable.

In addition, if the MCU clock system provides multiple clock sources for the peripherals, the peripherals can operate while the CPU is asleep. Consider an analogue-to-digital conversion that requires a high-speed clock. If the MCU clock system provides high-speed clock independently of a CPU, the CPU can sleep while the ADC is operational, saving current consumption of the CPU.

Interrupts. Interrupts bring an MCU out of an LPM. So the more the interrupts in an MCU, the more the flexibility to prevent current-wasting CPU polling and reduce power consumption. A good low-power MCU will have extensive interrupt capability, providing interrupts for all its peripherals and many external interrupts for external events. Not only does the polling itself use power but also controlling the polling interval requires a timer, which uses additional current.